

Lab5-demo

2026-05-12

This full data set includes more possible predictors than the text alone, but for this model we will only use the text variable

```
urlfile ="https://raw.githubusercontent.com/MaRo406/EDS-231-text-sentiment/main/data/climbing_reports_model_dat.csv"
incidents_df<-readr::read_csv(url(urlfile))
```

```
## Rows: 2770 Columns: 5
## — Column specification —————
## Delimiter: ","
## chr (3): ID, Accident Title, Text
## dbl (2): Publication Year, Deadly
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
glimpse(incidents_df)
```

```
## Rows: 2,770
## Columns: 5
## $ ID          <chr> "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", ...
## $ `Accident Title` <chr> "Failure of Rappel Setup (Protection Pulled Out), I...
## $ `Publication Year` <dbl> 1990, 1990, 1990, 1990, 1990, 1990, 1990, 1990, 199...
## $ Text        <chr> "Colorado, Rocky Mountain National Park\nOn May 7, ...
## $ Deadly      <dbl> NA, NA, NA, NA, 1, 1, NA, NA, NA, NA, NA, 1, 1, NA,...
```

How many incidents are marked as deadly?

```
incidents_df |>
  count(Deadly, name = "n")
```

```
## # A tibble: 2 × 2
##   Deadly      n
##   <dbl> <int>
## 1     1     563
## 2    NA    2207
```

Now we'll split our data into training and test portions

```
set.seed(321)
incidents2class <- incidents_df |>
  mutate(deadly = factor(if_else(
    is.na(Deadly),
    "non-deadly", "deadly"
  )))

#split our data
incidents_split <- initial_split(incidents2class, prop = 0.8, strata = deadly)

incidents_train <- training(incidents_split)
incidents_test  <- testing(incidents_split)
```

We use recipe() to specify the predictor and outcome variables and the data.

```
incidents_rec <- recipe(deadly ~ Text, data = incidents_train)
```

Next we add some familiar pre-processing steps on our Text variable: tokenize to word level, filter to the most common words, and calculate tf-idf.

```
recipe <- incidents_rec |>
  step_tokenize(Text) |>
  step_tokenfilter(Text, max_tokens = 1000) |>
  step_tfidf(Text)

recipe
```

```
##
```

```
## — Recipe —————
```

```
##
```

```
## — Inputs
```

```
## Number of variables by role
```

```
## outcome: 1
## predictor: 1
```

```
##
```

```
## — Operations
```

```
## • Tokenization for: Text
```

```
## • Text filtering for: Text
```

```
## • Term frequency-inverse document frequency with: Text
```

Python parallel: This three-step recipe maps almost directly onto a single sklearn object:

```
TfidfVectorizer(max_features=1000) .
```

```
nb_spec <- naive_Bayes() %>%
  set_mode("classification") %>% #set modeling context
  set_engine("naivebayes") #method for fitting model
```

Time for model evaluation. We'll do a standard resampling process to evaluate our naive Bayes model. Here we create 10-fold cross-validation sets, and use them to estimate performance. `vfold_cv` let's us do this.

```
set.seed(123)
incidents_folds <- vfold_cv(incidents_train)

incidents_folds
```

```
## # 10-fold cross-validation
## # A tibble: 10 × 2
##   splits      id
##   <list>    <chr>
## 1 <split [1993/222]> Fold01
## 2 <split [1993/222]> Fold02
## 3 <split [1993/222]> Fold03
## 4 <split [1993/222]> Fold04
## 5 <split [1993/222]> Fold05
## 6 <split [1994/221]> Fold06
## 7 <split [1994/221]> Fold07
## 8 <split [1994/221]> Fold08
## 9 <split [1994/221]> Fold09
## 10 <split [1994/221]> Fold10
```

Create a tidymodels workflow to combine the modeling components

Python parallel: A `workflow()` is the tidymodels equivalent of `sklearn.pipeline.Pipeline`. Both bundle preprocessing and a model estimator into a single object that can be fit, cross-validated, and used for prediction in one call.

```
nb_wf <- workflow() |>
  add_recipe(recipe) |>
  add_model(nb_spec)
```

To estimate its performance, we fit the model to each of the resampled folds, and then evaluate on the heldout part of each resampled fold.

```
nb_rs <- tune::fit_resamples(
  nb_wf,
  incidents_folds,
  control = control_resamples(save_pred=TRUE)
)
```

Python parallel: `fit_resamples()` is doing what `cross_validate(pipeline, X, y, cv=kfold, scoring=['accuracy','roc_auc'], return_estimator=True)` does in sklearn — fitting the full pipeline (preprocessing + model) on each training fold and scoring on the held-out fold.

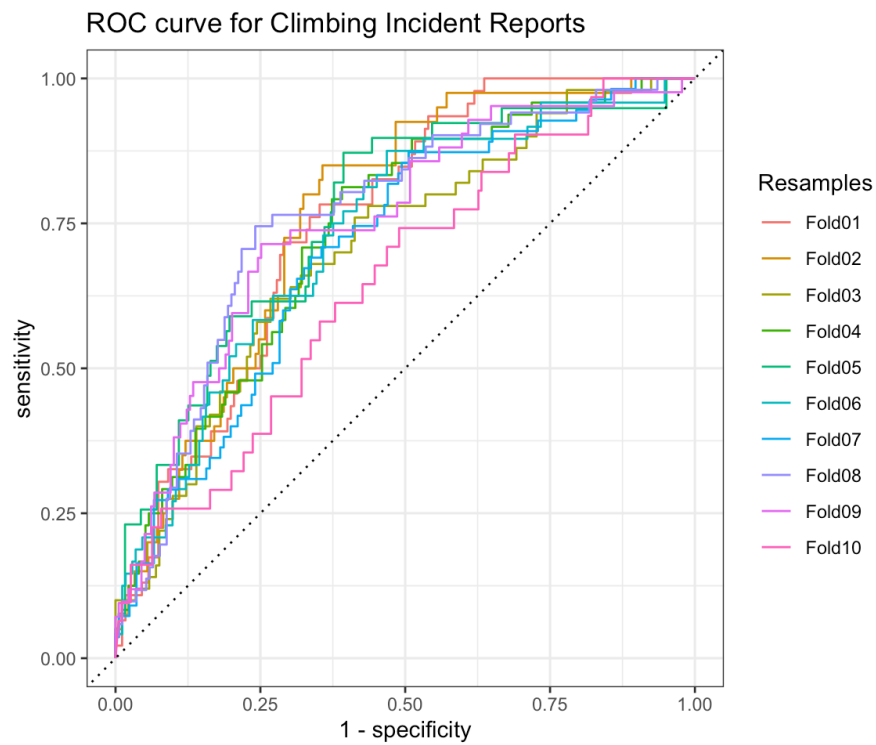
Extract the relevant information using `collect_metrics()` and `collect_predictions()` and examine the performance metrics.

```
nb_rs_metrics <- collect_metrics(nb_rs)
nb_rs_predictions <- collect_predictions(nb_rs)
nb_rs_metrics
```

```
## # A tibble: 3 × 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>    <dbl> <int>   <dbl> <chr>
## 1 accuracy binary    0.802   10 0.00986 pre0_mod0_post0
## 2 brier_class binary    0.198   10 0.00987 pre0_mod0_post0
## 3 roc_auc   binary    0.733   10 0.0114  pre0_mod0_post0
```

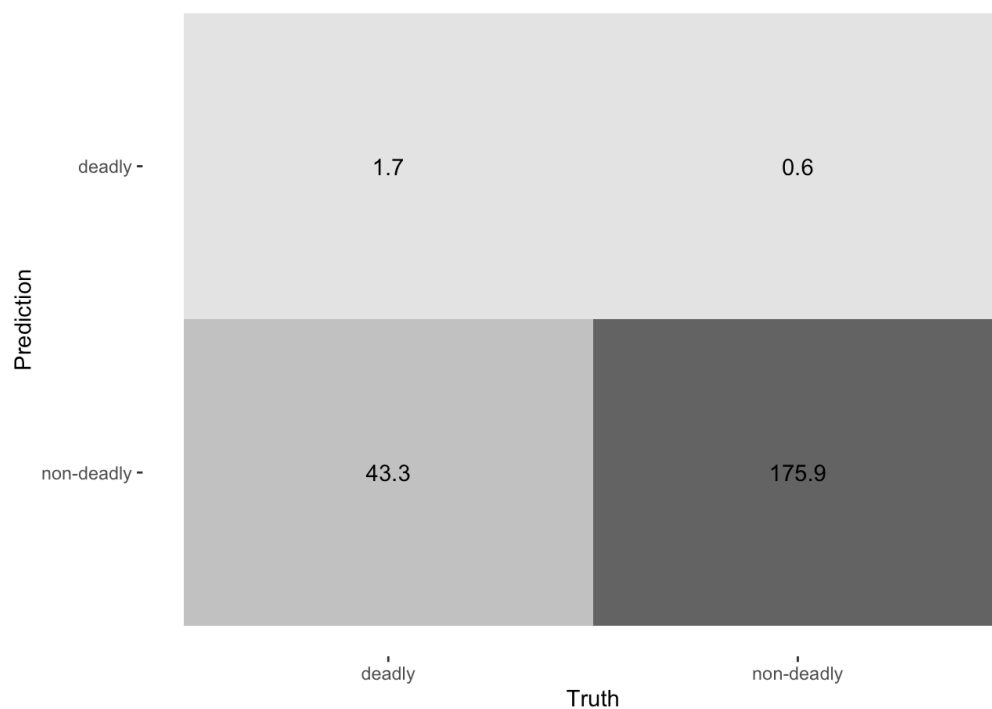
We'll use two performance metrics: accuracy and ROC AUC. Accuracy is the proportion of the data that is predicted correctly. The ROC curve plots the true positive rate against the false positive rate; AUC closer to 1 indicates a better-performing model, while AUC closer to 0.5 indicates a model that does no better than random guessing.

```
nb_rs_predictions %>%
  group_by(id) %>%
  roc_curve(truth = deadly, .pred_deadly) %>%
  autoplot() +
  labs(
    color = "Resamples",
    title = "ROC curve for Climbing Incident Reports"
  )
)
```



Another model method involves the confusion matrix. A confusion matrix tabulates a model's false positives and false negatives for each class.

```
conf_mat_resampled(nb_rs, tidy = FALSE) %>% #compute matrix for each fold then average
  autoplot(type = "heatmap")
```



Hmm, let's try a different model.

Lasso classification uses regularization on regression to produce a simpler, more generalizable model. It learns how much of a penalty to apply to features, reducing the high-dimensional space of tokens — sometimes shrinking coefficients all the way to 0, effectively removing those features from the model entirely.

The penalty is a hyperparameter we need to choose. Rather than setting it manually, we estimate its best value by fitting models across 30 candidate penalty values on our resampled data and comparing performance.

```
tune_spec <- logistic_reg(penalty = tune(), mixture = 1) %>%  
  set_mode("classification") %>%  
  set_engine("glmnet")  
  
tune_spec
```

```
## Logistic Regression Model Specification (classification)  
##  
## Main Arguments:  
##   penalty = tune()  
##   mixture = 1  
##  
## Computational engine: glmnet
```

```
tune_spec <- logistic_reg(penalty=tune(), mixture = 1) |>  
  set_mode("classification") |>  
  set_engine("glmnet")  
  
tune_spec
```

```
## Logistic Regression Model Specification (classification)  
##  
## Main Arguments:  
##   penalty = tune()  
##   mixture = 1  
##  
## Computational engine: glmnet
```

Here we use `grid_regular()` to create 30 possible values for the regularization penalty. Then `tune_grid()` fits a model at each of those values.

```
lambda_grid <- grid_regular(penalty(), levels = 30)  
  
lambda_grid
```

```
## # A tibble: 30 × 1  
##   penalty  
##   <dbl>  
## 1 1e-10  
## 2 2.21e-10  
## 3 4.89e-10  
## 4 1.08e- 9  
## 5 2.40e- 9  
## 6 5.30e- 9  
## 7 1.17e- 8  
## 8 2.59e- 8  
## 9 5.74e- 8  
## 10 1.27e- 7  
## # i 20 more rows
```

Python parallel: `grid_regular(penalty(), levels = 30) + tune_grid()` is akin to using `GridSearchCV()`.

```

lasso_wf <- workflow() %>%
  add_recipe(recipe) %>%
  add_model(tune_spec)

set.seed(2023)
tune_rs <- tune_grid(
  lasso_wf,
  incidents_folds,
  grid = lambda_grid,
  control = control_resamples(save_pred = T)
)

```

```
collect_metrics(tune_rs)
```

```

## # A tibble: 90 × 7
##   penalty .metric .estimator mean      n std_err .config
##   <dbl> <chr>      <chr>    <dbl> <int>  <dbl> <chr>
## 1 1 e-10 accuracy  binary  0.886   10 0.00502 pre0_mod01_post0
## 2 1 e-10 brier_class binary  0.102   10 0.00364 pre0_mod01_post0
## 3 1 e-10 roc_auc   binary  0.919   10 0.00554 pre0_mod01_post0
## 4 2.21e-10 accuracy  binary  0.886   10 0.00502 pre0_mod02_post0
## 5 2.21e-10 brier_class binary  0.102   10 0.00364 pre0_mod02_post0
## 6 2.21e-10 roc_auc   binary  0.919   10 0.00554 pre0_mod02_post0
## 7 4.89e-10 accuracy  binary  0.886   10 0.00502 pre0_mod03_post0
## 8 4.89e-10 brier_class binary  0.102   10 0.00364 pre0_mod03_post0
## 9 4.89e-10 roc_auc   binary  0.919   10 0.00554 pre0_mod03_post0
## 10 1.08e-9 accuracy  binary  0.886   10 0.00502 pre0_mod04_post0
## # i 80 more rows

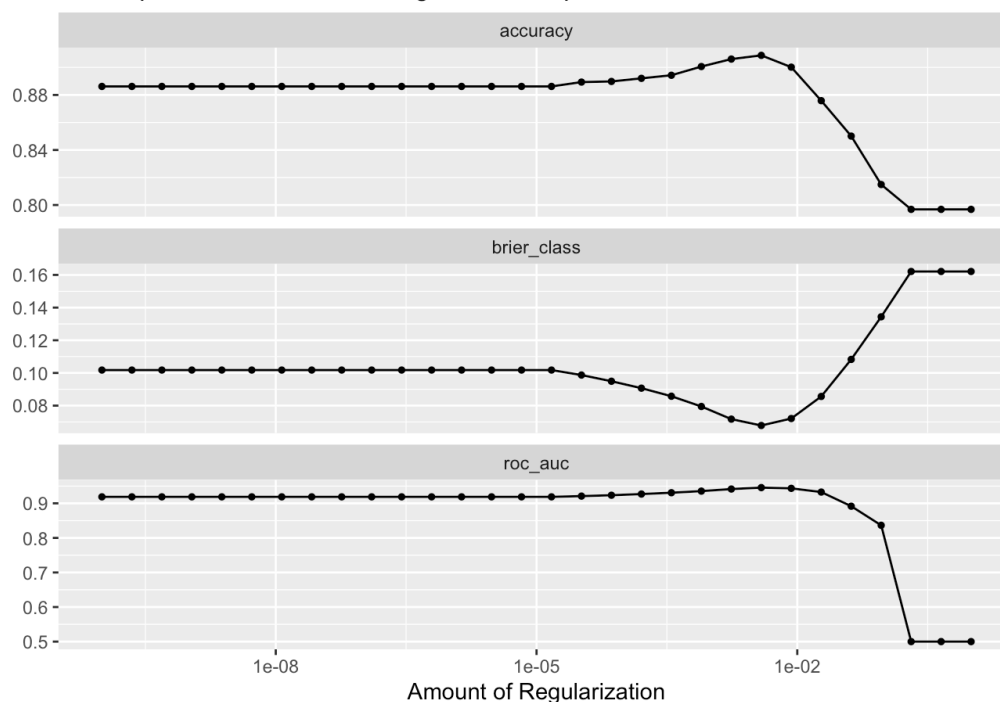
```

```

autoplot(tune_rs) +
  labs(
    title = "Lasso performance across regularization penalties"
  )

```

Lasso performance across regularization penalties



```

tune_rs %>%
  show_best(metric = "roc_auc")

```

```
## # A tibble: 5 × 7
##   penalty .metric .estimator mean      n std_err .config
##   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 0.00386 roc_auc binary    0.945   10 0.00489 pre0_mod23_post0
## 2 0.00853 roc_auc binary    0.943   10 0.00498 pre0_mod24_post0
## 3 0.00174 roc_auc binary    0.941   10 0.00491 pre0_mod22_post0
## 4 0.000788 roc_auc binary    0.935   10 0.00482 pre0_mod21_post0
## 5 0.0189  roc_auc binary    0.933   10 0.00609 pre0_mod25_post0
```

```
tune_rs %>%
  show_best(metric = "accuracy")
```

```
## # A tibble: 5 × 7
##   penalty .metric .estimator mean      n std_err .config
##   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 0.00386 accuracy binary    0.909   10 0.00471 pre0_mod23_post0
## 2 0.00174 accuracy binary    0.906   10 0.00492 pre0_mod22_post0
## 3 0.000788 accuracy binary    0.901   10 0.00492 pre0_mod21_post0
## 4 0.00853 accuracy binary    0.900   10 0.00557 pre0_mod24_post0
## 5 0.000356 accuracy binary    0.894   10 0.00566 pre0_mod20_post0
```

```
chosen_acc <- tune_rs %>%
  select_by_one_std_err(metric = "accuracy", -penalty)
```

Next, let's finalize our workflow with this particular regularization penalty. This is the regularization penalty that our tuning results indicate give us the best model.

```
final_lasso <- finalize_workflow(lasso_wf, chosen_acc
  )

final_lasso
```

```
## == Workflow ==
## Preprocessor: Recipe
## Model: logistic_reg()
##
## — Preprocessor —
## 3 Recipe Steps
##
## • step_tokenize()
## • step_tokenfilter()
## • step_tfidf()
##
## — Model —
## Logistic Regression Model Specification (classification)
##
## Main Arguments:
##   penalty = 0.00385662042116347
##   mixture = 1
##
## Computational engine: glmnet
```

The penalty argument value now reflects our tuning result. Now we fit to our training data.

```
fitted_lasso <- fit(final_lasso, incidents_train)

fitted_lasso
```

```

## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: logistic_reg()
##
## — Preprocessor —————
## 3 Recipe Steps
##
## • step_tokenize()
## • step_tokenfilter()
## • step_tfidf()
##
## — Model —————
##
## Call: glmnet::glmnet(x = maybe_matrix(x), y = y, family = "binomial",      alpha = ~1)
##
##      Df  %Dev  Lambda
## 1      0  0.00 0.156000
## 2      2  3.64 0.142200
## 3      2  6.63 0.129500
## 4      2  9.00 0.118000
## 5      2 11.01 0.107500
## 6      3 12.93 0.097990
## 7      3 15.19 0.089290
## 8      3 17.21 0.081350
## 9      3 18.99 0.074130
## 10     3 20.57 0.067540
## 11     4 22.15 0.061540
## 12     4 23.72 0.056070
## 13     6 25.34 0.051090
## 14     7 26.96 0.046550
## 15     9 28.64 0.042420
## 16    13 30.40 0.038650
## 17    15 32.30 0.035220
## 18    18 34.17 0.032090
## 19    25 36.36 0.029240
## 20    30 38.59 0.026640
## 21    35 40.77 0.024270
## 22    44 42.89 0.022120
## 23    51 45.01 0.020150
## 24    58 47.14 0.018360
## 25    64 49.17 0.016730
## 26    73 51.20 0.015240
## 27    79 53.10 0.013890
## 28    91 54.96 0.012660
## 29   104 56.81 0.011530
## 30   116 58.68 0.010510
## 31   121 60.48 0.009574
## 32   141 62.26 0.008723
## 33   166 64.12 0.007948
## 34   186 65.98 0.007242
## 35   204 67.81 0.006599
## 36   226 69.58 0.006013
## 37   243 71.32 0.005479
## 38   257 72.99 0.004992
## 39   272 74.61 0.004548
## 40   291 76.19 0.004144
## 41   304 77.71 0.003776
## 42   311 79.15 0.003441
## 43   324 80.55 0.003135
## 44   343 81.90 0.002857
## 45   362 83.18 0.002603
## 46   367 84.43 0.002372
##
## ...
## and 54 more lines.

```


First let's look at the words associated with an accident being non-deadly.

```
fitted_lasso |>
  extract_fit_parsnip() |>
  tidy() |>
  arrange(-estimate)
```

```
## # A tibble: 1,001 × 3
##   term                estimate penalty
##   <chr>                <dbl>   <dbl>
## 1 tfidf_Text_analysis    136.  0.00386
## 2 tfidf_Text_hospital    91.1  0.00386
## 3 tfidf_Text_injury      82.9  0.00386
## 4 tfidf_Text_condition   76.1  0.00386
## 5 tfidf_Text_treatment   70.7  0.00386
## 6 tfidf_Text_me          67.3  0.00386
## 7 tfidf_Text_assessment  65.4  0.00386
## 8 tfidf_Text_fractured   63.5  0.00386
## 9 tfidf_Text_pain        62.5  0.00386
## 10 tfidf_Text_case       59.4  0.00386
## # i 991 more rows
```

And now the words that are most associated with a deadly incident.

```
fitted_lasso|>
  extract_fit_parsnip() |>
  tidy() |>
  arrange(-estimate)
```

```
## # A tibble: 1,001 × 3
##   term                estimate penalty
##   <chr>                <dbl>   <dbl>
## 1 tfidf_Text_analysis    136.  0.00386
## 2 tfidf_Text_hospital    91.1  0.00386
## 3 tfidf_Text_injury      82.9  0.00386
## 4 tfidf_Text_condition   76.1  0.00386
## 5 tfidf_Text_treatment   70.7  0.00386
## 6 tfidf_Text_me          67.3  0.00386
## 7 tfidf_Text_assessment  65.4  0.00386
## 8 tfidf_Text_fractured   63.5  0.00386
## 9 tfidf_Text_pain        62.5  0.00386
## 10 tfidf_Text_case       59.4  0.00386
## # i 991 more rows
```

Finally, let's fit to the test data and see how we did.

```
last_fit(final_lasso, incidents_split) %>%
  collect_metrics()
```

```
## # A tibble: 3 × 4
##   .metric    .estimator .estimate .config
##   <chr>      <chr>      <dbl> <chr>
## 1 accuracy  binary         0.919 pre0_mod0_post0
## 2 roc_auc   binary         0.951 pre0_mod0_post0
## 3 brier_class binary         0.0620 pre0_mod0_post0
```

Lab 5 Assignment: Due May 19 at 11:59pm

Language & Tools

You may complete this assignment in **R** or **Python** — your choice. If you work in Python, submit a `.ipynb` or `.py` script along with the knit product instead of the `.Rmd`.

AI Use

You are permitted to use Claude (or another LLM) **more liberally than in previous labs** for this assignment, including for help writing or debugging code you haven't used before. However:

- **Every AI-assisted code block must include a comment** identifying it as AI-assisted and briefly describing what you asked for (e.g., `# Claude: asked for help adapting the tune_grid workflow to a random forest spec`).
- **All written interpretation must be your own.** Do not use AI to generate your written responses to the questions below — these are graded primarily on your reasoning, not on the code that produced the results.

Grading

Grading for this lab weights **interpretation and documentation more heavily than code correctness**. A working model with shallow interpretation will score lower than a model with minor implementation issues but clear, thoughtful commentary on what the results mean.

Tasks

1. Select a third classification algorithm beyond Naive Bayes and Lasso (e.g., random forest, k-nearest neighbors, support vector machine, decision tree). Report its cross-validated accuracy and ROC AUC.
2. Identify the key hyperparameter(s) for your chosen algorithm. Tune at least one hyperparameter using cross-validation and select a final value. Briefly explain in plain language what the hyperparameter controls and why tuning it matters.
3. Fit your final tuned model and evaluate it on the test set. Compare its accuracy and ROC AUC to the Naive Bayes and Lasso models from the demo. Which model performs best, and why do you think that is?
4. Examine the confusion matrix for your tuned model. What does the confusion matrix tell you about how the model is handling the minority class? Is overall accuracy a good summary metric here? What would you prioritize — minimizing false negatives (missed fatalities) or false positives (false alarms) — and why?